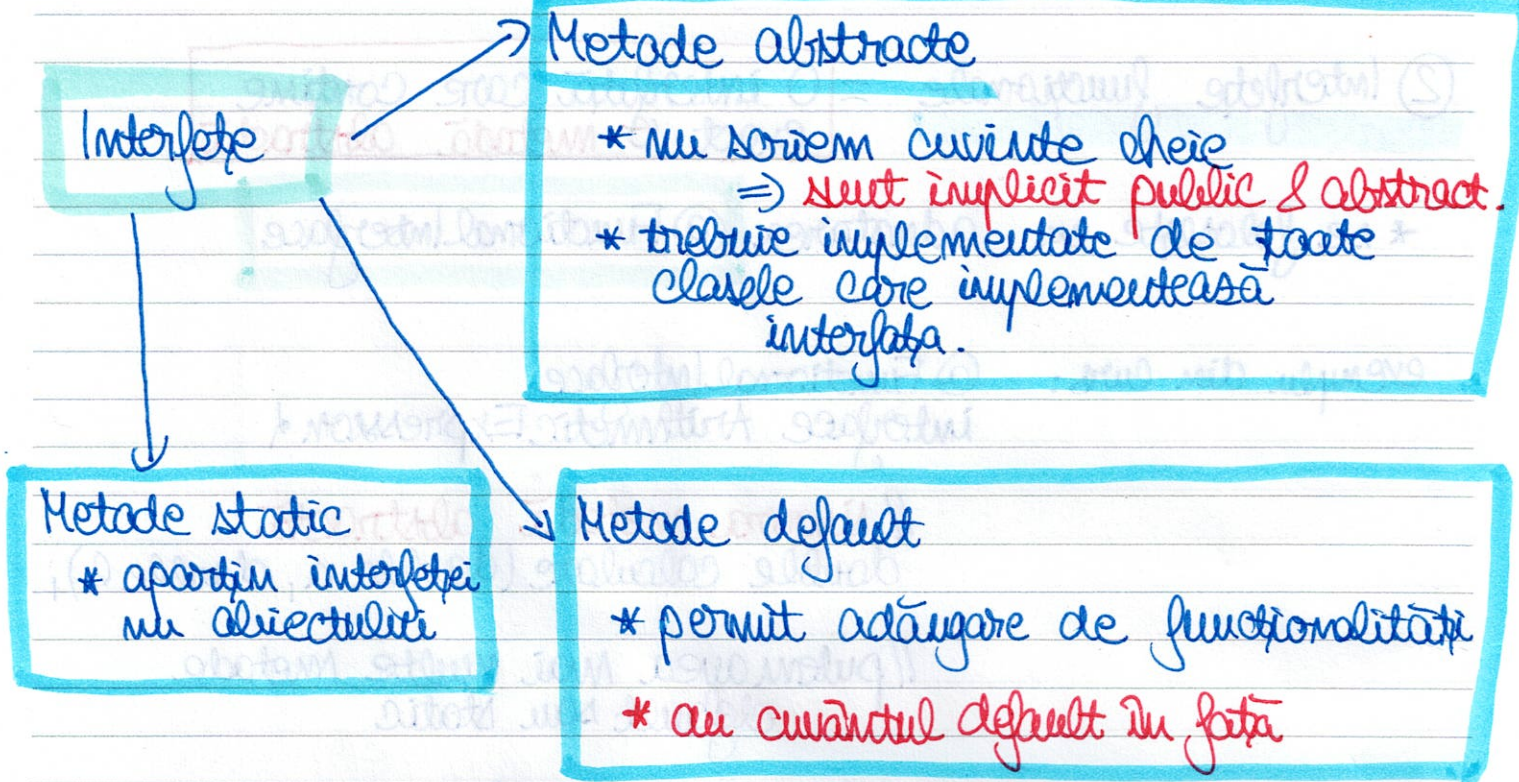


Capital: Interfețe funcționale / Operații pe stream-uri

① Metodele default și static în interfețe

→ până la Java 8, interfețele aveau "contracte" pur abstracte.

→ aveau doar semnături de metode, fără cod.



Exemplu din curs.

interface Arithmetic {

`double pi = 3.14;`

→ orice variabilă declarată într-o interfață este în mod implicit **public, static, final**.
final ⇒ valoarea ei nu poate fi schimbată.

`double calculate(double a, double b);`

→ **metodă abstractă**

`default double sqrt(double a) {
 return Math.sqrt(a);
}`

→ **metodă default** (are cod)

`static double suma(double a, double b) {
 return a + b;
}`

→ **metodă statică** (utilitară)

* Interfață:

* Metodele abstracte $\rightarrow$ obligatoriu `@Override`

* Metodele default $\rightarrow$ Ob, pot fi suprascrise

(interfața are o implementare de rezervă).

* Metodele statice $\rightarrow$ NU POT FI SUPRASCRISE

② Interfețe funcționale = O interfață care conține exact O metodă abstractă

* se folosește cu adnotarea `@FunctionalInterface`

exemplu din curs:

```
@FunctionalInterface
interface ArithmeticExpression {
```

```
// singura metodă abstractă
    double calculate(double a, double b);
```

```
// putem avea mai multe metode
    default sau static
```

```
    default void printSolut() {
        System.out.println("Solut");
    }
```

```
}
```

I. Referința la o metodă de clasă

dacă avem clasă

```
class FormulaHelper {
```

```
    public static double putrat
        (double x, double y) {
        return Math.pow(x+y, 2);
    }
```

```
}
```

Știm că interfața funcțională așteaptă o acțiune, iar mai avem deja o metodă statică într-o clasă care face exact acea acțiune (are **aceeași semnătură**).

→ `ArithmeticExpression b2 = FormulaHelper::patrat;`

//interfața cere `double calculate(double a, double b);`

⇒ apoi putem apela `double a = 2.1; double b = 2.2;`
`double rez = b2.calculate(a, b);`

II. Referința la o metodă de instanță a unui obiect specific

~~pasul 1: creăm obiectul care reține starea~~

~~`FormulaHelper helper = new FormulaHelper(a, b);`~~

```
class FormulaHelper {  
    private double a, b;  
    public FormulaHelper(double a, double b) {  
        this.a = a;  
        this.b = b;  
    }  
    public double distanta(double x, double y) {  
        return Math.Sqrt(-----)  
    }  
}
```

@FunctionalInterface

interface ArithmeticExpression {

double calculate(double a, double b);

}

↓
antecaptă o acțiune.

pasul 1: Creăm alieciul

FormulaHelper helper = new FormulaHelper(a, b);

pasul 2: folosim metoda "distanța" a acestui alieci specific.

~~Arith Expre~~

ArithmeticExpression ~~value~~ dist = helper::distanța;

pasul 3: apelul

double a = 3;

double b = 2.5;

double rez = dist.calculate(a, b);

III. Referința la constructor

```
interface StudentFactory <S extends Student> {  
    S create(int id, String nume, float media);  
}
```

// referințe la constructori

```
StudentFactory <Student> studentFactory = Student::new;  
studentFactory.create(1, "Pop", 8.5);
```

Tip Referință	Sintaxă	Ce face în spate (Lambda)
Statică	Clasa::metoda	(args) -> Clasa.metoda(args)
Instanță (obiect specific)	obiect::metoda	(args) -> obiect.metoda(args)
Instanță (obiect arbitrar)	Clasa::metoda	(obiect, args) -> obiect.metoda(args)
Constructor	Clasa::new	(args) -> new Clasa(args)